

Lab1 back-propagation

314551161 黃頌閣

1. Introduction

本次 Lab 的具體目標為理解並實作一個包含兩個隱藏層的簡單神經網路，完成前向傳播與反向傳播的完整流程，並使用這個自建的模型來解決 Linear 與 XOR 這兩種資料集的分類問題。實作上只使用 Numpy 與 Python 標準函式庫進行底層的矩陣運算與梯度推導，預期透過權重的更新，讓模型的預測準確率達到 90% 以上的標準。

2. Implementation Details

A. Sigmoid function

- 定義與特性：

Sigmoid 是一個呈現 S 型曲線的實數函數。它的主要作用是將神經元的線性輸出壓縮到 $(0, 1)$ 之間，藉此加入非線性特徵。其函數及導函數公式如下：

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- Sigmoid 程式碼實作：

```
class Sigmoid:
    def forward(self, x):
        return 1 / (1 + np.exp(-x))

    def derivative(self, x):
        return x * (1 - x)
```

B. Neural network architecture

- 架構定義：

依照實驗要求，架構包含一個輸入層、兩個隱藏層以及一個輸出層。為了增加模型的擬合能力，除了 Weight 矩陣外，每層皆額外引入了 bias 向量。其前向傳播的數學定義如下：

- 第一隱藏層： $Z_1 = \text{act}(X \cdot W_1 + b_1)$
- 第二隱藏層： $Z_2 = \text{act}(Z_1 \cdot W_2 + b_2)$
- 輸出層： $y_{\text{pred}} = \sigma(Z_2 \cdot W_3 + b_3)$ (註： X 為輸入特徵， act 為隱藏層配置的函數， σ 為輸出層固定使用的 Sigmoid 函數)

- 矩陣維度：

假設單次輸入的資料數量為 N ，特徵維度為 2，隱藏層神經元數量為 H ，輸出維度為 1，各

層參數的維度定義如下：

- 輸入資料: $X(N, 2) \rightarrow$ 第一層: $W_1(2, H), b_1(1, H) \rightarrow Z_1(N, H)$ 。
- 輸入資料: $Z_1(N, H) \rightarrow$ 第二層: $W_2(H, H), b_2(1, H) \rightarrow Z_2(N, H)$ 。
- 輸入資料: $Z_2(N, H) \rightarrow$ 第三層: $W_3(H, 1), b_3(1, 1) \rightarrow y_{pred}(N, 1)$ 。(註：在加法運算時，Numpy 會自動將 $(1, H)$ 的偏差值擴充至與資料同等的 (N, H) 維度)
- 程式碼實作：
透過常態分佈 `np.random.randn` 來初始化各層的 Weight 與 bias。為了提升程式碼的擴充性與可讀性，我將所有需要優化的參數與其對應的梯度，分別封裝為 `params_list` 與 `grads_list` 讓後續 Optimizer 進行迭代更新時的邏輯更加簡潔。
- NeuralNetwork 程式碼實作:

```
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, loss_fn,
activation):
        self.loss_fn = loss_fn
        self.act = activation
        self.out_act = Sigmoid()

        self.W1 = np.random.randn(input_size, hidden_size)
        self.W2 = np.random.randn(hidden_size, hidden_size)
        self.W3 = np.random.randn(hidden_size, output_size)

        self.b1 = np.random.randn(1, hidden_size)
        self.b2 = np.random.randn(1, hidden_size)
        self.b3 = np.random.randn(1, output_size)

        self.params_list = [self.W1, self.W2, self.W3, self.b1, self.b2,
self.b3]
        self.grads_list = [0] * len(self.params_list)

    def forward(self, X):
        self.z1 = self.act.forward(X @ self.W1 + self.b1)
        self.z2 = self.act.forward(self.z1 @ self.W2 + self.b2)
        self.y_pred = self.out_act.forward(self.z2 @ self.W3 + self.b3)
        return self.y_pred
```

C. Backpropagation

Backpropagation 是一種應用在神經網路中計算梯度的演算法，透過微積分的連鎖律來迭代計算每一層的梯度。本實驗採用最基礎的 Mean Squared Error 作為 Loss function $L(\theta)$ ，整個演算法可分為兩個主要部分 -- Propagation 以及 Weight update。

- Mean Squared Error：

- 函數公式： $L_{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$
- 導函數公式： $\frac{\partial L_{MSE}}{\partial \hat{y}_i} = \frac{1}{N} \frac{\partial L_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial y_i} = \frac{2}{N} (\hat{y}_i - y_i)$
- δ 計算：
在反向傳播階段，我們必須先計算預測值與真實標籤之間的誤差，並將此誤差逐層往回傳遞，以產生隱藏層與輸出層神經元的 δ 值。
 - 損失函數梯度 $\frac{\partial L}{\partial \hat{y}} = L'(\hat{y} - y)$
 - 輸出層誤差 $\delta_3 = \frac{\partial L}{\partial \hat{y}} \odot \sigma'(\hat{y})$
 - 第二隱藏層誤差 $\delta_2 = (\delta_3 \cdot W_3^T) \odot \sigma'(Z_2)$
 - 第一隱藏層誤差 $\delta_1 = (\delta_2 \cdot W_2^T) \odot \sigma'(Z_1)$
- Gradient 計算：
取得各層的誤差項 δ 後，我們將其與各層輸入值相乘，即可得到各層權重的梯度。
 - 輸出層梯度： $dW_3 = Z_2^T \cdot \delta_3$ $db_3 = \sum \delta_3$
 - 第二隱藏層梯度： $dW_2 = Z_1^T \cdot \delta_2$ $db_2 = \sum \delta_2$
 - 第一隱藏層梯度： $dW_1 = X^T \cdot \delta_1$ $db_1 = \sum \delta_1$
- Weight Update：
我們實作了 Stochastic gradient descent 作為優化器。其核心在於將計算出的梯度乘上 Learning rate 後，從原有的權重中減去。權重必須往梯度的反方向更新，才能有效降低模型的誤差。更新公式： $W_{new} = W_{old} - \eta \cdot dW$
- MSE 程式碼實作：

```
class MSE:
    def forward(self, y_pred, y_true):
        return np.mean((y_pred - y_true) ** 2)

    def derivative(self, y_pred, y_true):
        return 2 * (y_pred - y_true) / len(y_true)
```

- backward 程式碼實作:

```
def backward(self, X, y_true):
    dy_pred = self.loss_fn.derivative(self.y_pred, y_true)
    delta3 = dy_pred * self.out_act.derivative(self.y_pred)
    delta2 = (delta3 @ self.W3.T) * self.act.derivative(self.z2)
    delta1 = (delta2 @ self.W2.T) * self.act.derivative(self.z1)

    self.dW1 = X.T @ delta1
    self.dW2 = self.z1.T @ delta2
    self.dW3 = self.z2.T @ delta3
    self.db1 = np.sum(delta1, axis=0, keepdims=True)
    self.db2 = np.sum(delta2, axis=0, keepdims=True)
```

```
self.db3 = np.sum(delta3, axis=0, keepdims=True)

self.grads_list[:] = [self.dW1, self.dW2, self.dW3, self.db1, self.db2,
self.db3]
```

- SGD 程式碼實作:

```
class SGD:
    def __init__(self, model, lr):
        self.model = model
        self.lr = lr

    def step(self):
        for i, grad in enumerate(self.model.grads_list):
            self.model.params_list[i] -= grad * self.lr
```

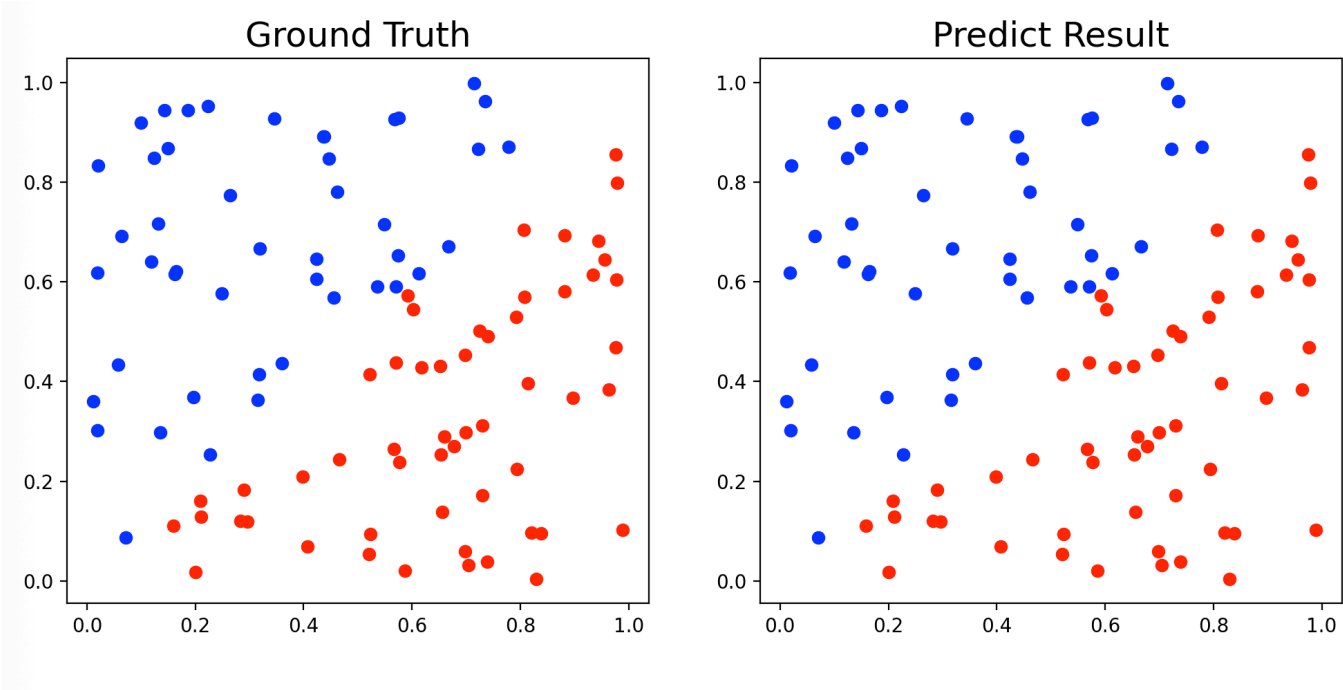
3. Experimental Results (for both datasets)

Setup

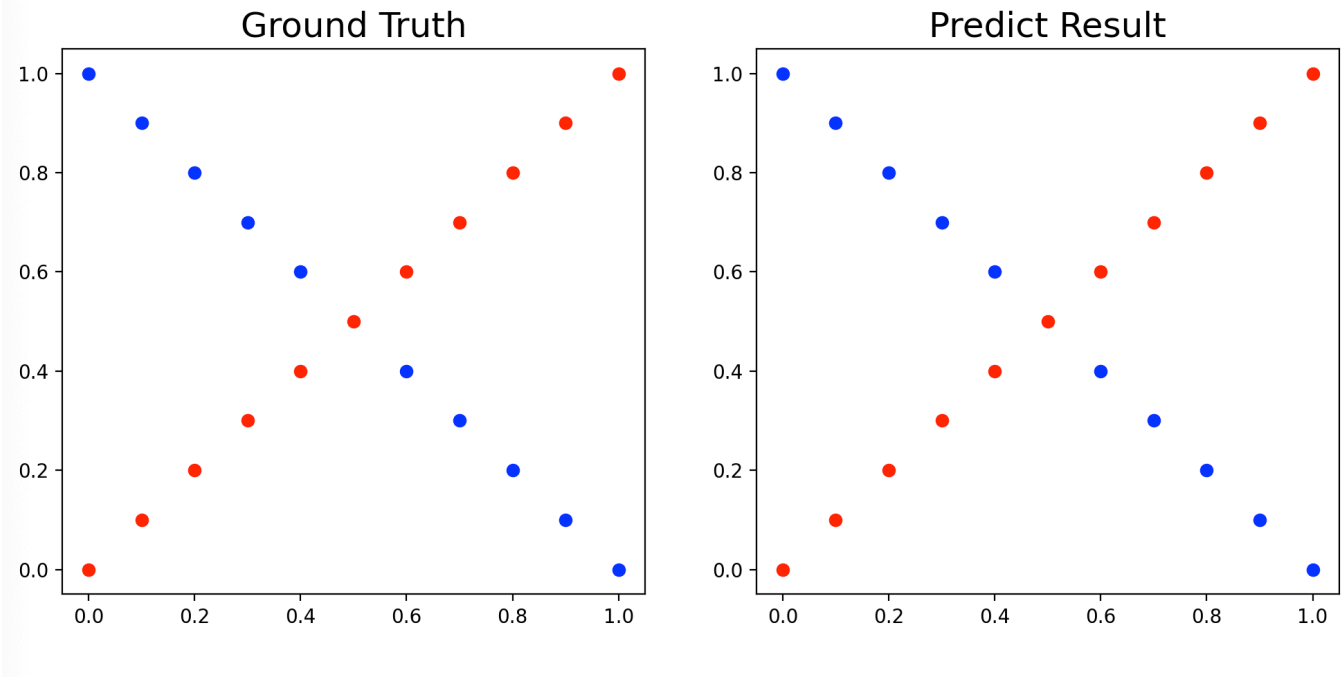
- epochs: 100000
- learning_rate: 0.1
- hidden_units: 4
- activation: Sigmoid
- loss_function: MSE
- optimizer: SGD

A. Screenshot and comparison figure

Linear Data



XOR Date



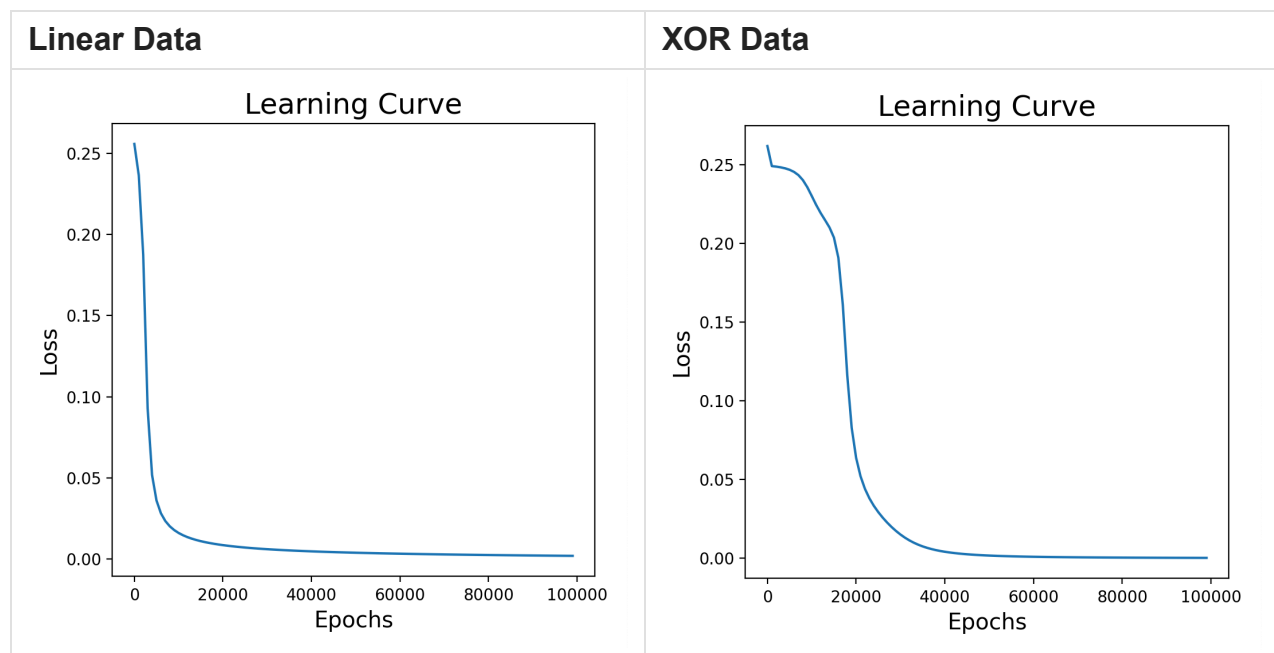
B. Show the accuracy of your prediction

(受限於圖片大小，因此只印出最後十個點)

Linear Data	XOR Data
<pre> point 91 : [ground truth : 1] [prediction : 1] point 92 : [ground truth : 0] [prediction : 0] point 93 : [ground truth : 0] [prediction : 0] point 94 : [ground truth : 1] [prediction : 1] point 95 : [ground truth : 0] [prediction : 0] point 96 : [ground truth : 0] [prediction : 0] point 97 : [ground truth : 1] [prediction : 1] point 98 : [ground truth : 0] [prediction : 0] point 99 : [ground truth : 1] [prediction : 1] point100 : [ground truth : 1] [prediction : 1] testing loss: 0.00196, accuracy: 100.00% </pre>	<pre> point 12 : [ground truth : 0] [prediction : 0] point 13 : [ground truth : 1] [prediction : 1] point 14 : [ground truth : 0] [prediction : 0] point 15 : [ground truth : 1] [prediction : 1] point 16 : [ground truth : 0] [prediction : 0] point 17 : [ground truth : 1] [prediction : 1] point 18 : [ground truth : 0] [prediction : 0] point 19 : [ground truth : 1] [prediction : 1] point 20 : [ground truth : 0] [prediction : 0] point 21 : [ground truth : 1] [prediction : 1] testing loss: 0.00030, accuracy: 100.00% </pre>

C. Learning curve (loss-epoch curve)

由實驗結果可見，XOR Data 學習速度上明顯較慢於 Linear Data，這也反映出非線性問題以及線性問題學習難度的差異。



D. Anything you want to present

Training Log

Linear Data	XOR Data
<pre> -----training part----- epoch 0 : [training loss : 0.26172] epoch 5000 : [training loss : 0.24667] epoch 10000 : [training loss : 0.23003] epoch 15000 : [training loss : 0.20363] epoch 20000 : [training loss : 0.06382] epoch 25000 : [training loss : 0.02934] epoch 30000 : [training loss : 0.01513] epoch 35000 : [training loss : 0.00760] epoch 40000 : [training loss : 0.00419] epoch 45000 : [training loss : 0.00260] epoch 50000 : [training loss : 0.00178] epoch 55000 : [training loss : 0.00130] epoch 60000 : [training loss : 0.00100] epoch 65000 : [training loss : 0.00081] epoch 70000 : [training loss : 0.00067] epoch 75000 : [training loss : 0.00056] epoch 80000 : [training loss : 0.00048] epoch 85000 : [training loss : 0.00042] epoch 90000 : [training loss : 0.00037] epoch 95000 : [training loss : 0.00033] </pre>	<pre> -----training part----- epoch 0 : [training loss : 0.25569] epoch 5000 : [training loss : 0.03610] epoch 10000 : [training loss : 0.01608] epoch 15000 : [training loss : 0.01103] epoch 20000 : [training loss : 0.00859] epoch 25000 : [training loss : 0.00710] epoch 30000 : [training loss : 0.00608] epoch 35000 : [training loss : 0.00533] epoch 40000 : [training loss : 0.00475] epoch 45000 : [training loss : 0.00429] epoch 50000 : [training loss : 0.00390] epoch 55000 : [training loss : 0.00358] epoch 60000 : [training loss : 0.00331] epoch 65000 : [training loss : 0.00307] epoch 70000 : [training loss : 0.00285] epoch 75000 : [training loss : 0.00267] epoch 80000 : [training loss : 0.00250] epoch 85000 : [training loss : 0.00234] epoch 90000 : [training loss : 0.00220] epoch 95000 : [training loss : 0.00208] </pre>

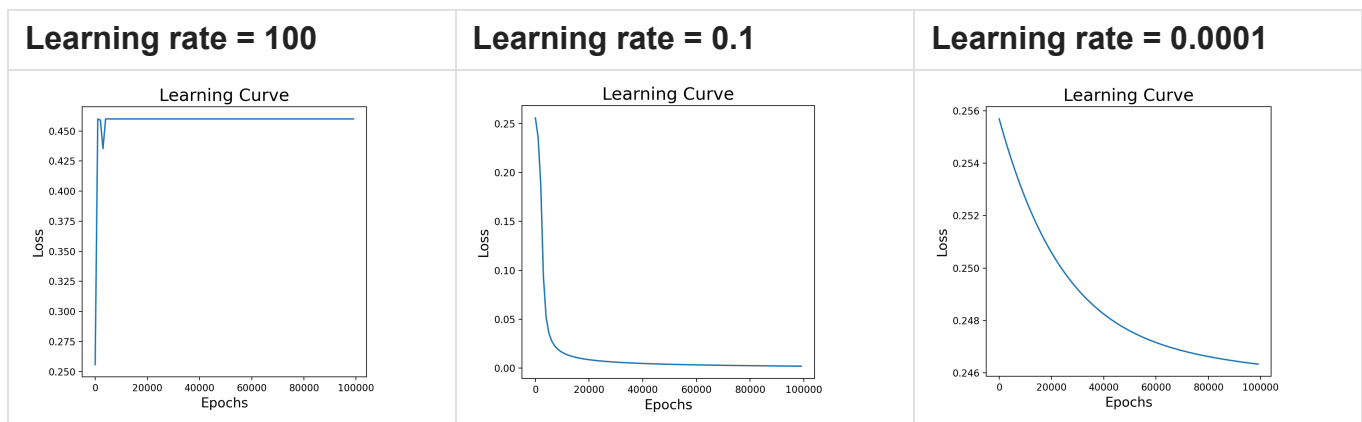
4. Discussion (for both datasets)

Setup

- epochs: 100000
- learning_rate: 0.1
- hidden_units: 4
- activation: Sigmoid
- loss_function: MSE
- optimizer: SGD

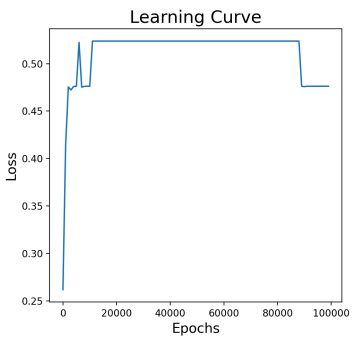
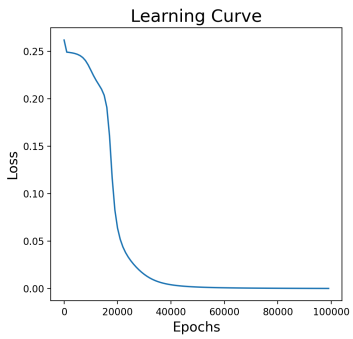
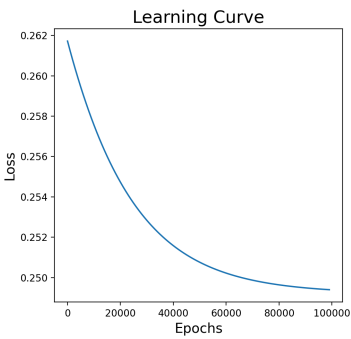
A. Try different learning rates

Linear Data



Learning rate = 100	Learning rate = 0.1	Learning rate = 0.0001
testing loss: 0.46000, accuracy: 54.00%	testing loss: 0.00196, accuracy: 100.00%	testing loss: 0.24632, accuracy: 54.00%

XOR Data

Learning rate = 100	Learning rate = 0.1	Learning rate = 0.0001
		
testing loss: 0.24939, accuracy: 52.38%	testing loss: 0.00030, accuracy: 100.00%	testing loss: 0.47619, accuracy: 52.38%

Learning Rate 決定了神經網路在計算出梯度後，進行權重更新時的步伐大小，它直接影響了模型學習的速度與品質。

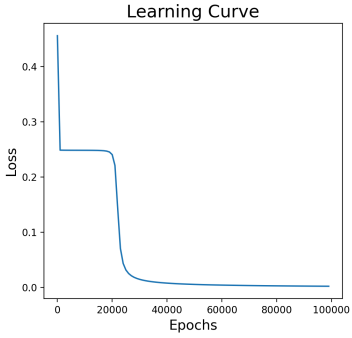
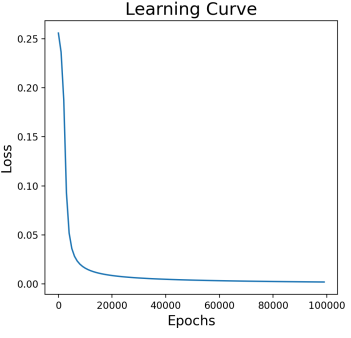
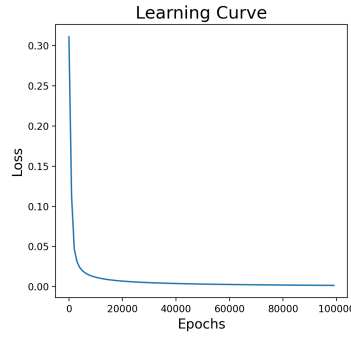
- 過大：

模型在誤差曲面上進行梯度下降時，步伐會邁得太大導致發生 Overshooting 的現象。這會反映在 Loss 曲線上出現劇烈的上下震盪，甚至最終發散導致模型完全無法收斂。實驗測試準確率僅落在 52% 到 54% 左右幾乎等同於隨機瞎猜，說明過大的學習率會直接破壞梯度的收斂軌跡。
- 過小：

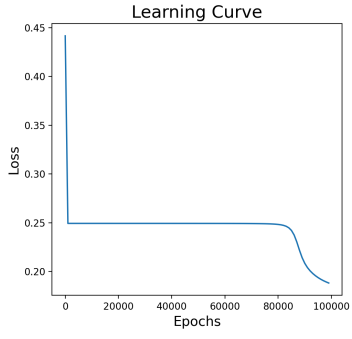
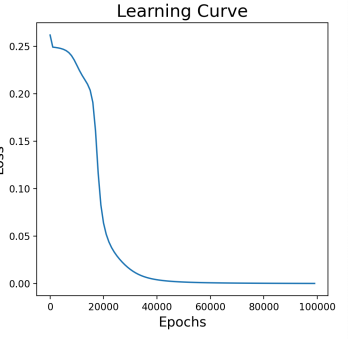
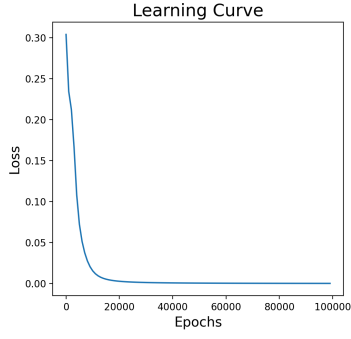
會導致收斂速度極度緩慢，在 Epoch 數量有限的情況下模型來不及走到谷底。在面對複雜的高維度 Error surface 時，過小的步伐極容易讓模型陷入 Local Minima 或 Saddle Points 中。實驗訓練結束時，Loss 來不及收斂仍停留在 0.24 與 0.47 的高點，導致最終準確率皆只有 50% 左右。

B. Try different numbers of hidden units

Linear Data

hidden units = 1	hidden units = 4	hidden units = 16
		
testing loss: 0.00239, accuracy: 100.00%	testing loss: 0.00196, accuracy: 100.00%	testing loss: 0.00161, accuracy: 100.00%

XOR Data

hidden units = 1	hidden units = 4	hidden units = 16
		
testing loss: 0.18728, accuracy: 71.43%	testing loss: 0.00030, accuracy: 100.00%	testing loss: 0.00023, accuracy: 100.00%

隱藏層神經元的數量直接決定了模型提取特徵的能力與特徵空間的維度。從實驗結果可以觀察到，不同複雜度的資料集對隱藏單元數量的需求截然不同。

- Linear Data :
 - 當 hidden units = 1 時，模型依然具備足夠的能力找出一條線性決策邊界，最終準確率達到 100%。但觀察其 Learning curve，Loss 在下降過程中出現了明顯的階梯狀停滯期。
 - 當 hidden units = 4 與 hidden units = 16 時，更多的神經元提供了更充裕的優化路徑，Loss 下降的坡度更陡峭，最終的測試誤差也進一步微幅降低。
- XOR Data :
 - 當 hidden units = 1 時，模型缺乏足夠的維度來建構非線性預測。Learning curve 顯示 Loss 長期卡在 0.25 左右無法下降，最終準確率也僅有 71.43%，證明了單一神經元無法解決 XOR 問題。

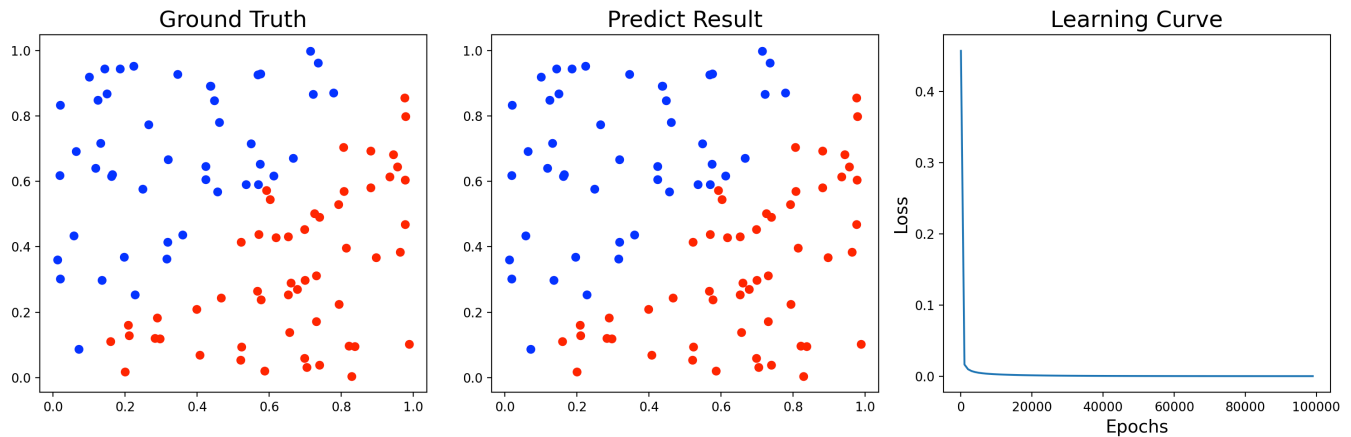
- 當 hidden units = 4 時，模型能達到 100% 準確率。若進一步擴大至 hidden units = 16，Loss 曲線在訓練極初期就呈現幾乎垂直的陡降。

對於複雜的非線性問題，必須賦予模型足夠的隱藏單元以確保擬合能力，但若是相對簡單的任務，過多的神經元雖然能加速收斂，在實務上也需考量運算資源的消耗。

C. Try without activation functions

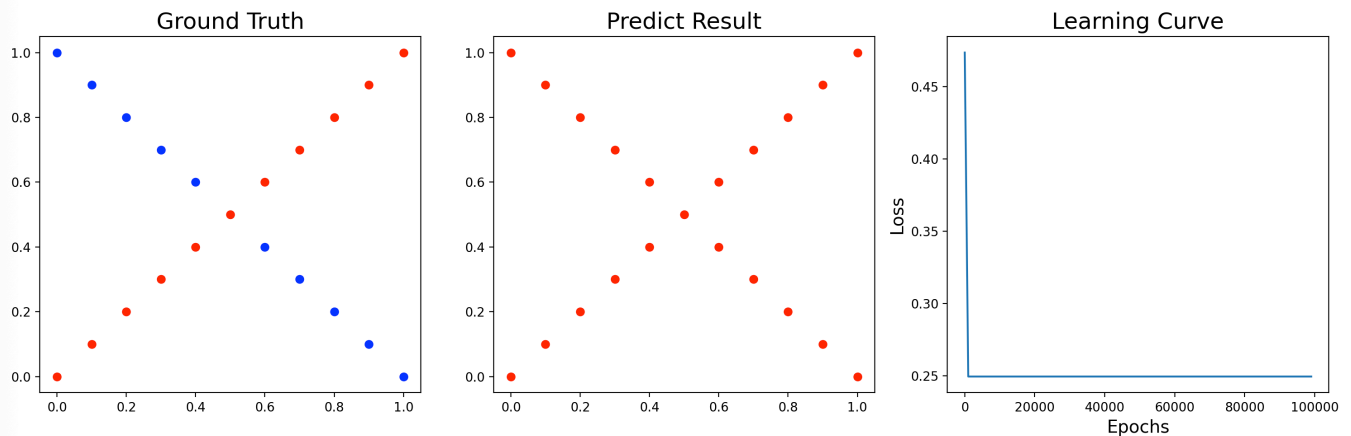
Linear Data

testing loss: 0.00008, accuracy: 100.00%



XOR Data

testing loss: 0.24943, accuracy: 52.38%



根據線性代數原理，多個線性矩陣相乘等價於一個單一矩陣的線性轉換，亦即 $X \cdot W_1 \cdot W_2 \cdot W_3$ 在數學上可簡化為 $X \cdot W_{eq}$ 。因此若將網路中所有的 Sigmoid 啟動函數移除，則會導致多層神經網路徹底退化。

- Linear 資料集：
因為目標本身只需一條直線切分，移除啟動函數後，網路退化成簡單的線性迴歸模型，依然能對 Linear 資料集進行精確分類。
- XOR 資料集：
模型在此將完全崩潰。XOR 資料集無法被任何單一條直線切分，失去了啟動函數所提供的非

線性映射能力，無論疊加多少個隱藏層模型都只能畫出直線。在此情況下，XOR 的預測準確率將永遠卡在 50% 左右。

- NoActivation 程式碼實作：

```
class NoActivation:
    def forward(self, x):
        return x

    def derivative(self, x):
        return np.ones_like(x)
```

D. Anything you want to share

Try Binary Cross-Entropy

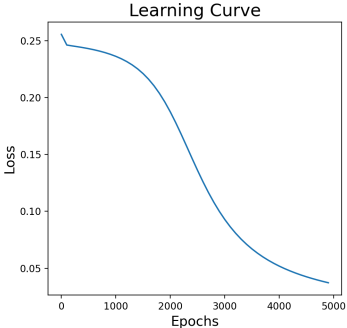
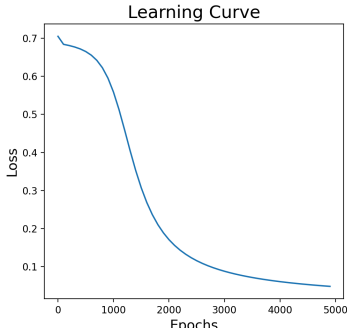
雖然本次作業的基礎規範是使用 Mean Squared Error 作為 Loss function，但理論上針對這類輸出為 0 或 1 的二元分類問題，使用 Binary Cross-Entropy 其實是最標準且合適的選擇。

- 函數公式： $L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
- 導函數公式： $\frac{\partial L_{BCE}}{\partial \hat{y}_i} = \frac{1}{N} \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}$
- BCE 程式碼實作：

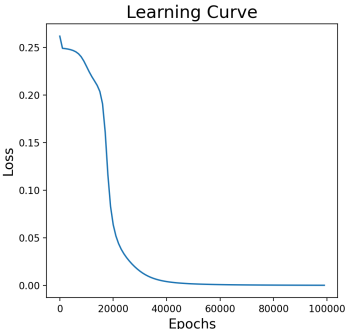
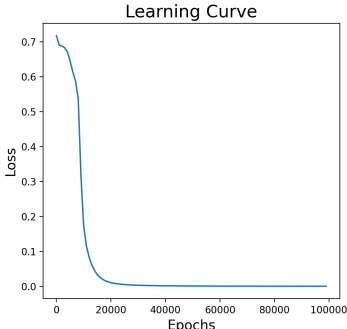
```
class BCE:
    def forward(self, y_pred, y_true):
        y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)
        return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

    def derivative(self, y_pred, y_true):
        y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)
        return ((y_pred - y_true) / (y_pred * (1 - y_pred))) / len(y_true)
```

Linear Data

MSE	BCE
 Learning Curve Loss Epochs	 Learning Curve Loss Epochs
testing loss: 0.00196, accuracy: 100.00%	testing loss: 0.00210, accuracy: 100.00%

XOR Data

MSE	BCE
 Learning Curve Loss Epochs	 Learning Curve Loss Epochs
testing loss: 0.00030, accuracy: 100.00%	testing loss: 0.00038, accuracy: 100.00%

從上方對比圖表可以明顯看出，兩者雖然皆能在訓練過後達到100%的精準率，但使用 Binary Cross-Entropy 比起 Mean Squared Error 在收斂速度上能有更好的表現。

5. Questions

A. What is the purpose of activation functions?

Activation functions 的核心目的是為神經網路引入非線性的元素，如果只使用線性函數，根據矩陣乘法的結合律，無論網路疊加了多少個隱藏層，最終的數學運算都可以被簡化合併為單一的線性方程式。這會導致神經網路徹底退化成簡單的線性迴歸模型，永遠無法解決像 XOR 這種非線性的問題。

此外，在處理二分類問題時，模型最後一層的輸出通常是一個不受限的實數。這對後續決策來說很難直觀判斷。使用 Sigmoid 函數將任何輸入壓縮到 $(0, 1)$ 區間。這讓輸出的結果可以被直觀地解釋為機率。

B. What might happen if the learning rate is too large or too small?

Learning Rate 決定了神經網路在計算出梯度後，進行權重更新時的步伐大小，它直接影響了模型學習的速度與品質。

- 過大：
模型在誤差曲面上進行梯度下降時，步伐會邁得太大導致發生 Overshooting 的現象。這會反映在 Loss 曲線上出現劇烈的上下震盪，甚至最終發散導致模型完全無法收斂。
- 過小：
會導致收斂速度極度緩慢，在 Epoch 數量有限的情況下模型來不及走到谷底。在面對複雜的高維度 Error surface 時，過小的步伐極容易讓模型陷入 Local Minimum 或 Saddle Point 中。

C. What is the purpose of weights and biases in a neural network?

Weights 與 Biases 是神經網路能夠學習並擬合任意函數的兩大核心參數，它們在幾何空間中扮演著互補的角色。

- Weights：
矩陣乘法 $X \cdot W$ 負責對輸入的特徵空間進行線性轉換，藉此找出特徵間的交互關係，這決定了各個輸入特徵對於該神經元的影響力或重要性。
- Biases：
如果網路中沒有 bias，所有分類的決策邊界都將被迫穿過原點，這會限制模型的擬合能力。加入 bias，模型才能靈活地在空間中的任意位置切分資料。

6. Extra

A. Implement different optimizers.

Adam

- 定義與特性：
 - Adam 結合了 Momentum 與 RMSProp 兩者的優勢，藉此為網路中的每一個參數提供自適應的 Learning Rate。讓模型在面對複雜的非線性地形時，展現出更加快速穩定的收斂能力。
 - Momentum：計算梯度的指數加權移動平均。
 - RMSProp：計算梯度平方的指數加權移動平均。

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- 偏差校正：修正訓練初期 m_t 與 v_t 容易趨近於 0 的偏差。

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

- 權重更新：結合上述結果進行參數 θ 的更新。

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

(註： t 為時間、 g_t 為當前參數梯度、 η 為學習率、 ϵ 為避免除以零的極小值常數)

- Adam 程式碼實作：

```
class Adam:
    def __init__(self, model, lr):
        self.model = model
        self.lr = lr
        self.beta1 = 0.9
        self.beta2 = 0.999
        self.beta1_t = self.beta1
        self.beta2_t = self.beta2
        self.m = [np.zeros_like(p) for p in model.params_list]
        self.v = [np.zeros_like(p) for p in model.params_list]

    def step(self):
        for i, grad in enumerate(self.model.grads_list):
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grad
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * (grad **
2)

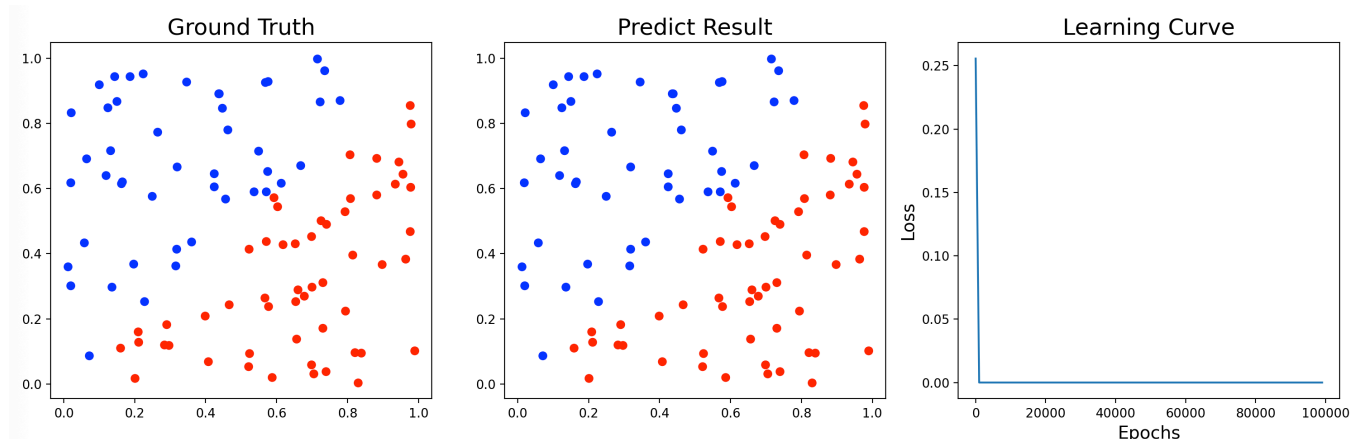
            m_head = self.m[i] / (1 - self.beta1_t)
            v_head = self.v[i] / (1 - self.beta2_t)
            self.model.params_list[i] -= self.lr * m_head / (np.sqrt(v_head)
+ 1e-5)

            self.beta1_t *= self.beta1
            self.beta2_t *= self.beta2
```

Linear Data Result

(Sigmoid + MSE + Adam , 4 hidden units)

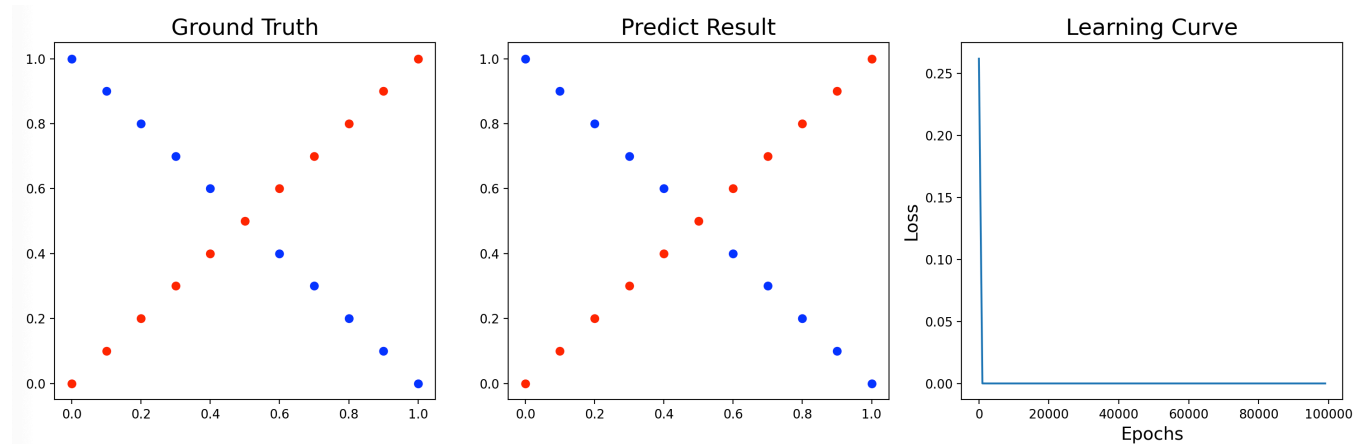
testing loss: 0.00000, accuracy: 100.00%



XOR Data Result

(Sigmoid + MSE + Adam, 4 hidden units)

testing loss: 0.00000, accuracy: 100.00%



B. Implement different activation functions.

ReLU

- 定義與特性：

ReLU 的數學定義極為乾淨俐落，相較 Sigmoid 省去了複雜的自然指數 e^{-x} 運算開銷。

$$f(x) = \max(0, x) \quad f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

- ReLU 程式碼實作：

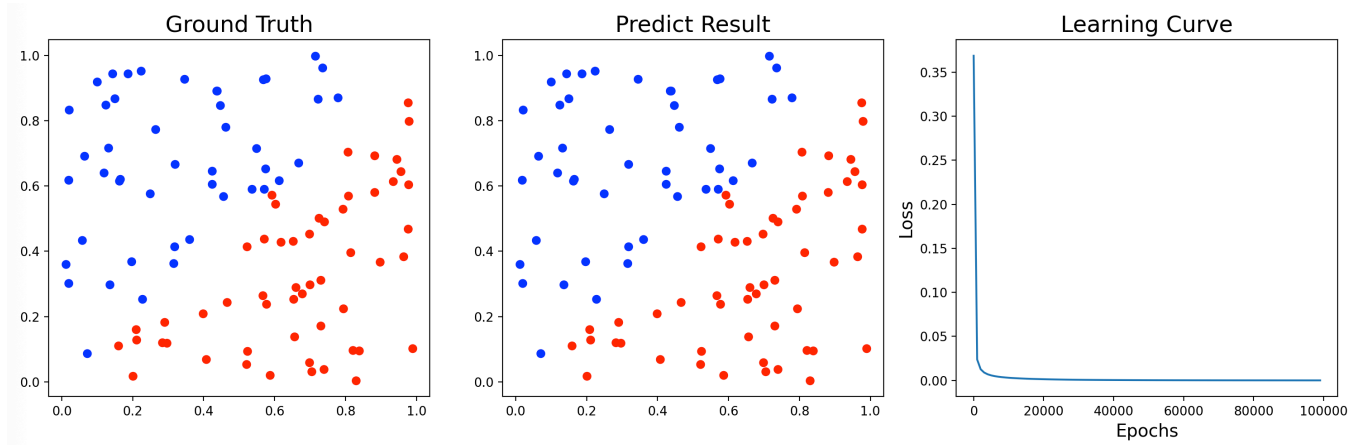
```
class ReLU:
    def forward(self, x):
        return np.maximum(0, x)

    def derivative(self, x):
        return (x > 0).astype(float)
```

Linear Data Result

(ReLU + MSE + SGD , 4 hidden units)

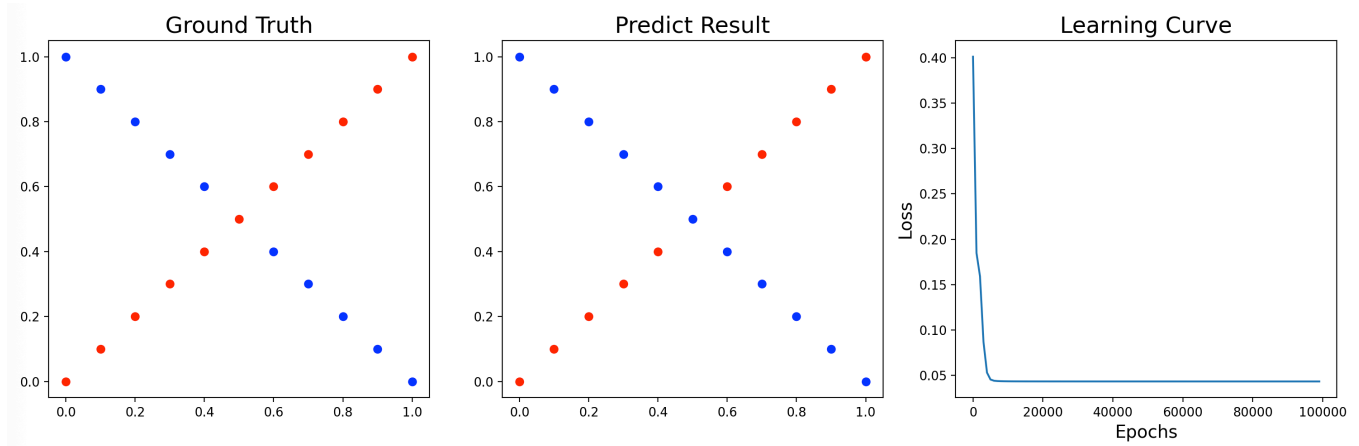
testing loss: 0.00011, accuracy: 100.00%



XOR Data Result

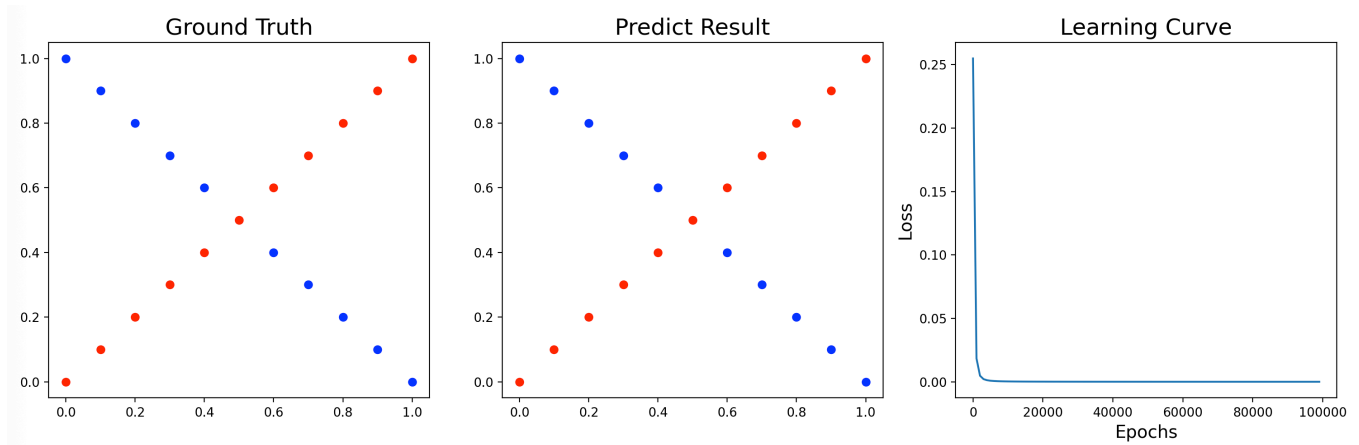
(ReLU + MSE + SGD , 4 hidden units)

testing loss: 0.04330, accuracy: 95.24%



(ReLU + MSE + SGD , 8 hidden units)

testing loss: 0.00001, accuracy: 100.00%



C. Implement convolutional layers.